MODULE 12 · DOMAIN 2 - IMPLEMENT GENERATIVE AI AND AGENTIC SOLUTIONS

Tuning generation

System messages, prompt techniques, sampling parameters, reasoning effort and budgets, and structured outputs.

DOMAIN 2 - Generative AI and agents WEIGHT 30-35% MODULE 12 of 39

admin@sthithapragna.com

Prompts say **what** you want. Parameters shape **how** the model samples. Structured outputs **guarantee** the shape.

Six levers, six jobs

LEVER	CONTROLS
System or developer message	Role, rules, tone, output format
Few-shot examples	Style, labels and patterns to copy
Grounding content	The facts the answer should use
Temperature or top_p	How varied the wording is
Reasoning effort	How much a reasoning model thinks
Structured outputs	The exact JSON shape returned

ON THE EXAM

Match the lever to the symptom. Wrong facts need grounding, not a lower temperature.

admin@sthithapragna.com

Designing the system message

- Define the **role and boundaries**: what the assistant does, and what it declines.
- Set **tone** and the **output format** you expect.
- **Give the model an out:** say what to reply when the answer isn't available, such as 'not found'.
- Avoid **conflicting instructions**, like 'be brief' and 'be comprehensive' with no priority.
- For reasoning models, a **developer message** works the same way as a system message.

> ON THE EXAM
>
> Instructions steer behaviour. As module 10 showed, they are a job description, not a lock.

Prompt techniques that work

- **Be specific** and restrict the space: 'three bullet points, each under twenty words'.
- **Ground the answer** in source material that is close to the final answer's form.
- **Break complex tasks into steps**, such as extract first, then summarise.
- **Few-shot examples** teach format and labels better than description. Their order matters.
- **Order matters generally:** instructions before or after long content change the result.

> ON THE EXAM
>
> Specific beats clever. Most prompt fixes are removing ambiguity, not adding magic words.

Where should instructions go?

INSTRUCTIONS ONLY AT THE START

Buried under content

- Long document follows the rules
- Later content has more influence
- Rules like word limits get ignored

INSTRUCTIONS REPEATED AT THE END

Doubled down

- Rules stated before the content
- Key rules repeated after it
- Last words reinforce the format

THE TIEBREAKER

Microsoft calls this doubling down: give instructions before and after your primary content.

Zero-shot, few-shot and chain of thought

STYLE	WHAT YOU PROVIDE	SUITS
Zero-shot	Just the instruction	Simple, well-known tasks
Few-shot	The instruction plus worked examples	A specific format, tone or label set
Chain of thought	A request to reason step by step	Multistep problems on standard models
Reasoning model	Goal, constraints, output contract	Hard problems; don't script every step

ON THE EXAM

Reasoning models already think before answering. Give them the goal and the rules, not a step-by-step script.

Parameters for standard models

PARAMETER	EFFECT	TYPICAL USE
`temperature`	More varied wording as it rises	Low for extraction
`top_p`	Limits choices to likely tokens	Tune this or temperature
`Max output tokens`	Caps response length	Stops runaway answers
`Penalties`	Less repetition, more new topics	Long creative output
`stop sequences`	End at a marker	Clean cut-off points

ON THE EXAM

The usual guidance is to tune temperature or top_p, but not both at once, so effects stay predictable.

Temperature zero makes output **more consistent**, not **more correct**. It can't add missing facts or enforce a format.

admin@sthithapragna.com

Parameters reasoning models reject

- **Not supported:** temperature, top_p, frequency and presence penalties, logit_bias, logprobs.
- **max_tokens isn't supported** either.
- Cap output with **max_output_tokens** in Responses, or **max_completion_tokens** in Chat Completions.
- Control thinking with **reasoning_effort** instead, and response length with verbosity where supported.
- Sending an unsupported parameter fails the request, so upgrades can break existing code.

> THE SIGNAL
>
> 'Upgraded to a reasoning model and every request now fails' usually means leftover temperature or max_tokens.

Budgeting for reasoning tokens

- Reasoning tokens are **invisible, but take up context and are billed as output**.
- The output cap covers **reasoning plus visible text**, together.
- Hit the cap and the response is **incomplete**, sometimes with **no visible answer at all**.
- Microsoft suggests **reserving at least 25,000 tokens** while you learn a workload's needs.
- Check the response **status**, and reasoning token counts in the usage details.

> ON THE EXAM
>
> An 'empty answer' from a reasoning model is often status incomplete, with the budget spent on thinking.

Choosing a reasoning effort

EFFORT	BEST FOR
`none`	Latency-critical work: voice, fast retrieval, classification
`low`	Tool use and planning where speed and cost matter
`medium`	A balanced starting point for most workloads
`high`	Hard reasoning, complex debugging, deep planning
`xhigh or max`	Deep research and long agentic runs, when evaluations justify it

ON THE EXAM

Supported values and defaults vary by model. Treat effort as a tuning knob, not the first fix when quality drops.

JSON mode or structured outputs?

JSON MODE

Valid JSON, any shape

- Guarantees syntactically valid JSON
- No schema: fields can be missing or extra
- Messages must ask the model for JSON

STRUCTURED OUTPUTS

Your schema, exactly

- Output follows the JSON Schema you supply
- Set strict to true
- Every field required; no additional properties
- Field order follows the schema

THE TIEBREAKER

Microsoft's own advice: if you need schema guarantees, switch to structured outputs.

Structured outputs in the Responses API

```
schema = {"type": "object",
    "properties": {
        "invoice_no": {"type": "string"},
        "total": {"type": "number"},
        "po_number": {"type": ["string", "null"]}},
    "required": ["invoice_no", "total", "po_number"],
    "additionalProperties": False}

response = client.responses.create(model="extract-prod", input=invoice_text,
    text={"format": {"type": "json_schema", "name": "invoice",
                     "strict": True, "schema": schema}})
```

ON THE EXAM

Optional fields are still required, typed as a union with null.

Structured output constraints

- A **subset** of JSON Schema: the root object can't be an anyOf.
- Up to **100 object properties** and **five levels** of nesting.
- Optional fields must still be required, typed with a **null union**.
- Not supported with the **bring-your-own-data** feature.
- A schema guarantees structure. Your code should still check business rules, like totals adding up.

> ON THE EXAM
>
> Deeply nested or open-ended schemas may need flattening before strict mode accepts them.

Symptom to lever

SYMPTOM	LEVER
Wording varies too much between runs	Lower temperature, or top_p
Code can't parse the output reliably	Structured outputs with a strict schema
Rules ignored in a long prompt	Repeat key instructions after the content
Invented answers when information is missing	Grounding, plus an explicit 'not found' out
Reasoning model slow on simple questions	Lower reasoning effort
Empty answer, status incomplete	Raise the output token cap

ON THE EXAM

Each row is a different lever. Diagnose the symptom first, then choose the lever that matches it.

A team upgrades a Chat Completions app's deployment from a standard model to a reasoning model, to improve answers on complex questions.

Immediately, every request fails with an error about an unsupported parameter. The code sets temperature to 0.2 and max_tokens to 800.

What should the team change?

A Set temperature to 0, which every model accepts, and keep max_tokens.

B Replace temperature with top_p set to 0.2, and keep max_tokens.

C Request a quota increase for the new reasoning model deployment.

D Remove temperature, and use max_completion_tokens not max_tokens.

◆ PAUSE AND DECIDE

What should the team change?

A Temperature 0, keep max_tokens.

Reasoning models reject temperature at any value, and max_tokens too.

B Swap to top_p.

top_p is also unsupported, and max_tokens still fails.

C Quota increase.

Quota problems return 429, not an unsupported parameter error.

D Drop temperature; use max_completion_tokens.

Removes both unsupported parameters and uses the cap reasoning models accept.

THE SIGNAL

'Upgraded, then unsupported parameter' is the leftover-parameter pattern. Control depth with reasoning_effort instead.

> An accounts app extracts invoice data with JSON mode and writes it straight into a database. The output is always valid JSON.
>
> But some responses omit the total field, and others add unexpected keys, which makes the database load fail.

What should the team change?

A Use structured outputs with a strict schema: all fields, no extras.

B Keep JSON mode, and add three examples of correctly formatted invoices.

C Keep JSON mode, and set temperature to zero for more consistent output.

D Keep JSON mode, and raise the maximum output tokens so no field is cut off.

◆ PAUSE AND DECIDE

What should the team change?

A Structured outputs, strict schema.

Every field required and no additional properties: the exact shape is guaranteed.

B JSON mode plus examples.

Examples improve the odds, but JSON mode still guarantees no schema.

C JSON mode, temperature zero.

More consistent, not guaranteed. A missing field can still be consistently missing.

D JSON mode, more output tokens.

Nothing is being truncated. The fields are omitted or invented, not cut off.

THE TIEBREAKER

Valid JSON is not the same as your schema. Only structured outputs guarantee the fields.

> A policy assistant's prompt begins with 'Answer in British English, in under 150 words'. A twenty-page policy document follows, then the user's question.
>
> Answers are accurate, but they often run to 400 words and use American spelling.

What should the team change first?

A Raise the temperature so the model explores more ways of phrasing answers.

B Repeat the length and spelling rules after the document, near the end.

C Switch to a reasoning model and set reasoning effort to high.

D Move the whole policy document into the system message instead.

◆ PAUSE AND DECIDE

What should the team change first?

A. Raise temperature.
Adds variety, which makes rule-following less consistent, not more.

B. Repeat the rules at the end.
Counters recency bias: the rules are now close to where the answer is produced.

C. Reasoning model, high effort.
Costly and slow, and nothing suggests the problem is reasoning ability.

D. Document into the system message.
Moves the content, but rules are still buried behind twenty pages.

THE SIGNAL

Accurate content with ignored formatting rules, in a long prompt, is recency bias. Double down.

> A research tool uses a reasoning model with max_output_tokens set to 2,000. Simple questions work well.
>
> On complex questions, users sometimes receive empty answers. Logs show those responses have status incomplete, with reason max_output_tokens.

What should the team do?

A Set reasoning effort to none so that no tokens go on thinking at all.

B Set temperature higher so the model reaches a visible answer sooner.

C Raise max_output_tokens to leave room for reasoning; check status.

D Retry the same request unchanged until a visible answer comes back.

◆ PAUSE AND DECIDE

What should the team do?

A Reasoning effort none.

Avoids the cap, but removes the thinking complex research questions need.

B Higher temperature.

Reasoning models don't support temperature, and it wouldn't shorten thinking.

C Raise the cap; check status.

The budget covers reasoning plus answer. A larger cap leaves room for both.

D Retry unchanged.

Hits the same cap again, and pays for the reasoning tokens every time.

THE SIGNAL

Empty answers on hard questions only, with status incomplete, means the thinking used up the budget.

Tuning generation in five lines

- **Prompts say what; parameters shape how; structured outputs guarantee the shape.**
- **Give an out, be specific, and repeat key rules after long content.**
- **Temperature zero is consistent, not correct.** Change temperature or top_p, not both.
- **Reasoning models reject temperature, top_p and max_tokens.** Budget for reasoning tokens.
- **JSON mode is valid JSON; structured outputs are your schema.** Every field required.

> ON THE EXAM
>
> Next, module 13: retrieval-augmented generation in an application, end to end.